

Порядок загрузки модулей

Приложения на Wasaby Framework построены по плагинной архитектуре. Функционал поставляется небольшими частями — модулями. Один модуль может опираться на функционал другого модуля т.е. находится от него в зависимости. Подробнее о зависимостях читайте [здесь](#).

Процедура загрузки модулей

Загрузка модулей при старте приложения происходит в строго определенной последовательности, которая состоит из следующих фаз:

1. Анализ зависимостей.
2. Загрузка библиотек в адресное пространство (только для модулей на C++).
3. Настройка платформоспецифичного окружения (iOS / Android).
4. Инициализация ядра.
5. Инициализация прикладных модулей.

Анализ зависимостей

При старте приложения возникает задача установления зависимостей между модулями, их инициализации и построения порядка загрузки. Способ поиска модулей для загрузки определяется значением параметра конфигурации **Модули**.

- Если параметр не задан, то модули берутся из каталога modules рядом с библиотекой sbis-lib.
- Если в качестве параметра задан путь к директории на диске, то будет производиться поиск файлов с расширением s3mod по подкаталогам верхнего уровня.
- Если указан путь к s3srv-файлу, то список модулей берется из него.

После получения списка модулей ядро сервиса определяет зависимости между модулями и вычисляет порядок загрузки. Если нет прямых и косвенных зависимостей, то порядок загрузки относительно друг друга не определен и может меняться от версии к версии платформы.

Загрузка библиотек в адресное пространство (C++)

На данном этапе происходит загрузка библиотек для модулей на C++ в адресное пространство в порядке зависимостей (в порядке загрузки, определенном в ходе предыдущей фазы).



Предупреждение

В точке подъема библиотек в адресное пространство платформа не готова к работе, так как её подсистемы не инициализированы. Поэтому исполнять сложную логику в этот момент времени нельзя.

Настройка платформоспецифичного окружения (iOS, Android)

На мобильных платформах существует дополнительная фаза, которая происходит между подъемом библиотек в адресное пространство и инициализацией ядра – настройка нативного окружения.

Ряд служб можно реализовать только с помощью нативного платформоспецифичного кода (Objective-C для iOS, Kotlin на Android). Для этого применяется инверсия зависимостей т.е. внутри модулей контроллера и ядра платформы объявляется интерфейс, а реализация подгружается из нативной части. Например, текущий язык системы можно узнать с помощью средств Android SDK.

iOS

Для iOS инициализаторы поставляются вместе с контроллером и их можно реализовать с помощью кода на Objective-C. Для этого необходимо создать наследника класса EnvironmentFillingCallback в Objective-C.

У класса `EnvironmentFillingCallback` есть два виртуальных метода `Name` и `Execute`. С помощью метода `Name` осуществляется поиск обработчика по имени модуля. В качестве возвращаемого значения выступает имя модуля. Посредством метода `Execute` производится настройка окружения.

```
1 class LD_DLLCLASS EnvironmentFillingCallback
2 {
3     public:
4         virtual ~EnvironmentFillingCallback();
5
6         // Выполнение настроек окружения.
7         virtual void Execute() const = 0;
8
9         // Получение имени модуля, которому соответствует обработчик.
10        virtual StringView Name() const noexcept = 0;
11 };
```

Собственную реализацию необходимо зарегистрировать с помощью макроса `REGISTER_ENVIRONMENT_FILLING_CALLBACK`,

```
1 class I18nEnvironmentFillingCallback : public
  ld::EnvironmentFillingCallback
2 {
3     ...;
4 }
5
6 REGISTER_ENVIRONMENT_FILLING_CALLBACK( I18nEnvironmentFillingCallback
  );
```

а также необходимо определить функцию `bool HasNativeInitializer() const noexcept override` с возвращаемым значением `True` в инициализаторе модуля.

```
1 namespace ld
2 {
3     class LD_DLLCLASS ModuleInitializerHandle : public AbstractModuleHandle
4     {
5     public:
6         bool HasNativeInitializer() const noexcept override
7         {
8             return true;
9         }
10    };
```

Android

В платформе есть специальный модуль `CoreInitializer`, который управляет вызовами Kotlin-инициализаторов модулей. После получения списка модулей через рефлексию происходит поиск классов с именами вида: `ru.tensor.Module$ИмяМодуляInitializer`. Например, для модуля `Dbi` ищется класс-инициализатор `ru.tensor.ModuleDbiInitializer`. Для всех найденных классов вызывается статический метод `init` в порядке загрузки модулей.

Для этого в каталоге проекта создайте следующую структуру файлов:

```
|— <каталог модуля>
|   |— java
|       |— ru
|           |— tensor
|               |— Module<Имя модуля>Initializer.kt
```

В файле `Module<Имя модуля>Initializer.kt` создайте класс инициализатора модуля с названием, соответствующим имени файла, и функцией `init`:

```

1 package ru.tensor;
2 import android.content.Context;
3 class ModuleDbiInitializer {
4     companion object {
5         @JvmStatic fun init(context: Context) {
6             // код инициализации модуля
7         }
8     }
9 }

```

Этот файл необходимо включить в список исходников [контроллера](#) в gradle-файле.

```

1 def javaSrcDirs = [ "${current_script_dir}/../sbis-dbi/java", ...]
2
3 sourceSets.main {
4     java.srcDirs += javaSrcDirs
5     ...
6 }

```

Также необходимо определить функцию `bool HasNativeInitializer() const noexcept override` с возвращаемым значением `True` в инициализаторе модуля.

```

1 namespace ld
2 {
3     class LD_DLLCLASS ModuleInitializerHandle : public AbstractModuleHandle
4     {
5     public:
6         bool HasNativeInitializer() const noexcept override
7         {
8             return true;
9         }
10 };

```

Инициализация модулей ядра

Модули ядра — это модули платформы. Для инициализации модулей ядра предназначены события `OnEndLoadKernelModules` и `OnKernelCleanup`.

Прикладным разработчикам запрещено объявлять коллбэки `OnEndLoadKernelModules` и `OnKernelCleanup`.



Предупреждение

Методы `OnEndLoadKernelModules` и `OnKernelCleanup` разрешено определять только разработчикам платформы и больше никому! В Python они недоступны.

События:

OnEndLoadKernelModules (C++) - происходит после получения списка инициализаторов. Вызывается для каждого модуля в прямом порядке зависимостей — от базовых модулей к высокоуровневым.

OnKernelCleanup (C++) - отвечает за освобождение ресурсов, задействованных после загрузки модуля. Вызывается для каждого модуля, в обратном порядке зависимостей — от высокоуровневых к базовым модулям.

Инициализация прикладных модулей

OnEndLoadModules/OnEndAllLoadModules (C++ / Python) происходит после загрузки всех модулей конкретного сервиса приложения в прямом порядке зависимостей — от базовых модулей к высокоуровневым.

OnCleanup (C++/Python) - отвечает за освобождение ресурсов, задействованных после загрузки модуля. Функция вызывается для каждого модуля, в обратном порядке — от высокоуровневых к базовым модулям.

Динамический инициализатор

Для того чтобы встроиться в процесс загрузки приложения, необходимо реализовать и зарегистрировать собственный динамический обработчик. Реализация собственных обработчиков подробно описана в статье ["События сервисного модуля и их обработчики"](#).